

UNITÉ 2 · SEMAINE 2

Appels système et E/S de bas niveau

📅 Semaine 2 · dates à confirmer </> Laboratoire TD1 (corrigé + défi IA) ~ 4 animations

LABORATOIRE TD1 · CORRIGÉ

Lire et écrire avec `read` et `write`

Les fondements de la programmation système: comment un programme entre dans le noyau, comment écrire et lire en binaire, et pourquoi il faut vérifier chaque résultat.

✓ **Devoir : rapport du TP**

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

EXERCICES

- | | | | |
|---|---|---|--|
| 1 | Comment un appel système est implémenté | 4 | <code>perror</code> est-il un appel système? |
| 2 | <code>echo</code> avec <code>write</code> | 5 | Copier avec <code>read</code> |
| 3 | Écrire un entier en binaire | 6 | Relire l'entier |
| | | 7 | Structures en binaire et remplissage |

TD1 · 1

● Corrigé · sortie réelle

Comment un appel système est implémenté

Énoncé. Un appel système nécessite une instruction spéciale du processeur (`syscall` sur x86-64, `svc` sur ARM, `ecall` sur RISC-V). Désassemblez la bibliothèque C avec `objdump -d` et trouvez comment le programme indique au noyau *quel* appel il veut. Puis observez avec `strace` les appels que fait votre `echo`.

Solution



Sortie réelle

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A4 -E '^[0-9a-f]+ <getppid@>'
0000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Sortie réelle

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A6 -E '^[0-9a-f]+ <__write_nocancel@>'
0000000000129060 <__write_nocancel@GLIBC_PRIVATE>:
    129060:    endbr64
    129064:    mov     $0x1,%eax
    129069:    syscall
    12906b:    cmp     $0xffffffffffffffff,%rax
    129071:    ja      129078 <__write_nocancel@GLIBC_PRIVATE+0x18>
    129073:    ret
```

Le numéro de l'appel va dans le registre `eax` (110 = `getppid`, 1 = `write` sur x86-64) et les arguments dans `rdi`, `rsi`, `rdx` ... L'instruction `syscall` fait passer le processeur en mode noyau et saute à l'entrée du noyau, qui utilise `eax` comme indice dans sa table des appels système. Au retour, le résultat est dans `rax` : une valeur entre -4095 et -1 signale une erreur, et l'enveloppe de la glibc la transforme en `-1 + errno` (c'est ce que fait `cmp $0xffffffffffffffff,%rax`).

Sortie réelle

```
$ strace -e trace=write ./echo hola mundo > /dev/null
write(1, "hola", 4)                = 4
write(1, " ", 1)                  = 1
write(1, "mundo", 5)              = 5
write(1, "\n", 1)                 = 1
+++ exited with 0 +++
```

Quatre appels `write` : un par mot, un pour l'espace et un pour le saut de ligne. Chacun est un aller-retour vers le noyau; dans l'unité 4, vous verrez comment la bibliothèque standard regroupe les écritures dans un tampon pour les économiser.

TD1 · 2

● Corrigé · testé

echo avec **write(2)**

Énoncé. Écrivez un programme **echo** qui copie ses arguments sur la sortie standard en utilisant seulement **write(2)**. Il doit vérifier les erreurs de **write** avec **exit_if**. Testez-le avec **./echo blabla < . 1>&0**.

Solution



C tds/td1/echo.c

```

/*
 * TD1 - exercise 3: echo with write(2).
 * Copies its arguments to standard output, separated by spaces, followed by
 a newline.
 * Try the error path with: ./echo blabla < . 1>&0 (stdout becomes a
 directory: write fails)
 */
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    for (int i = 1; i < argc; i++) {
        size_t len = strlen(argv[i]);
        ssize_t rc = write(STDOUT_FILENO, argv[i], len);
        exit_if(rc == -1, "write");
        /* write may write less than asked (pipe or socket full): it is not
 an error,
            but this program must then say so. write_all() in libutils loops
 instead. */
        exit_if((size_t)rc != len, "write: partial write");

        if (i < argc - 1)
            exit_if(write(STDOUT_FILENO, " ", 1) == -1, "write space");
    }
    exit_if(write(STDOUT_FILENO, "\n", 1) == -1, "write newline");
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./echo hola mundo
hola mundo

```

Sortie réelle

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

`< .` ouvre le répertoire courant comme entrée standard (descripteur 0) et `1>&0` fait de la sortie (1) une copie du descripteur 0. Un répertoire ouvert en lecture seule ne peut pas être écrit: `write` renvoie `-1` avec `errno = EBADF`, et `exit_if` le signale.

TD1 · 3

● Corrigé · testé

Écrire un entier en binaire

Énoncé. Écrivez `write_number` : il écrit sur la sortie standard le contenu mémoire d'un `long` valant 5, puis le nombre d'octets écrits par le premier appel. Que voyez-vous dans le terminal? Regardez la sortie avec `od`.

Solution



C tds/td1/write_number.c

```
/*
 * TD1 - exercise 4: write the memory content of a long, then the number of
 * bytes written.
 * The output is binary, not text: look at it with ./write_number | od -A d
 * -t x1
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    long value = 5;
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);    /* 8 bytes on
x86-64 */
    exit_if(written == -1, "write value");

    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00 00
00000016
```

Rien de lisible n'apparaît dans le terminal: les octets `05 00 00 00 00 00 00 00` ne sont pas des caractères imprimables. `od` montre la vérité: 8 octets du `long` dans l'ordre *little-endian* (octet de poids faible en premier, comme un processeur x86-64 range les entiers) et 8 octets contenant 8, ce qu'a renvoyé le premier `write`. `write` ne convertit rien en texte: il copie des octets.

TD1 · 4

● Corrigé

perror est-il un appel système?

Réponse

Non. `perror` est dans la section 3 du manuel: c'est une fonction de la bibliothèque C qui lit `errno`, cherche le texte de l'erreur et l'écrit sur l'erreur standard... en utilisant, en interne, l'appel système `write`. Vous pouvez le vérifier avec

```
strace ./echo blabla < . 1>&0 : un write(2, "write: Bad file descriptor\n", ...)
```

apparaît.

TD1 · 5

● Corrigé · testé

Copier l'entrée vers la sortie avec `read(2)`

Énoncé. Écrivez `read` : il lit l'entrée standard avec `read` dans un tableau de taille `BUFFER_SIZE` et l'écrit sur la sortie avec `write`, jusqu'à la fermeture de l'entrée (`Ctrl + D`). Testez-le comme `cat`, en copiant un fichier, et avec un répertoire en entrée.

Solution

C tds/td1/read.c

```

/*
 * TD1 - exercise 6: copy standard input to standard output with
 * read(2)/write(2).
 * Stops at end of file (Ctrl-D in a terminal). Usages:
 * ./read          ./read < file          ./read < in > out
 * ./read < . (error)
 */
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#ifndef BUFFER_SIZE
#define BUFFER_SIZE 4096
#endif

int main(void)
{
    char buffer[BUFFER_SIZE];

    for (;;) {
        ssize_t n = read(STDIN_FILENO, buffer, BUFFER_SIZE);
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "read");
        if (n == 0) /* end of file: the writer closed its
side */
            return EXIT_SUCCESS;

        /* write can be partial: loop until the n bytes are out (see
write_all in libutils). */
        ssize_t done = 0;
        while (done < n) {
            ssize_t rc = write(STDOUT_FILENO, buffer + done, (size_t)(n -
done));
            exit_if(rc == -1, "write");
            done += rc;
        }
    }
}

```

Sortie réelle

```
$ ./read < input.txt
first line
second line
```

Sortie réelle

```
$ ./read < binary.bin > copy.bin && cmp binary.bin copy.bin && echo identical
identical
```

Sortie réelle

```
$ ./read < .
read: Is a directory
[exit status 1]
```



Deux détails qui séparent un programme correct d'un programme qui marche « presque toujours »

- `read` renvoie **0** en fin de fichier, **-1** en cas d'erreur et sinon *jusqu'à* `BUFFER_SIZE` octets: il peut en renvoyer moins (un tube, un terminal ou un signal écourtent la lecture).
- `write` peut aussi écrire moins que demandé: c'est pourquoi la boucle interne recommence jusqu'à ce que les `n` octets soient sortis. Si un signal l'interrompt avant d'avoir rien écrit, il échoue avec `EINTR` et il suffit de réessayer.

TD1 · 6

● Corrigé · testé

Lire l'entier de `write_number`

Énoncé. Écrivez un programme qui lit ce que produit `write_number` et l'affiche avec `printf`.

Solution



C tds/td1/read_number.c

```

/*
 * TD1 - exercise 7: read what write_number produces and print it as text.
 * ./write_number | ./read_number      →  value = 5, bytes written = 8
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

/* read() may return fewer bytes than asked (e.g. from a pipe): loop until
`len` bytes. */
static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    long value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("value = %ld, bytes written = %zd\n", value, written);
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./write_number | ./read_number
value = 5, bytes written = 8

```

`read_exact` répète `read` jusqu'à obtenir exactement les octets demandés: à travers un tube, les 16 octets pourraient arriver en deux morceaux. Cela marche parce que les deux programmes tournent sur la même machine: ils partagent la taille de `long` et l'ordre des octets. Entre machines différentes, il faudrait convenir d'un format (par exemple l'ordre réseau avec `htonl`, unité 8).

TD1 · 7

● Corrigé · testé

Structures en binaire et remplissage

Énoncé. Transformez les programmes précédents en `write_struct` et `read_struct` pour une `struct nopad { char c1; long l; char c2; }` déclarée dans `utils.h`, contenant 'a', 5 et 'b'. Quelle est la taille de la structure? Change-t-elle avec `-O3`? Comment la rendre plus petite?

Solution

```
C tds/td1/write_struct.c

/*
 * TD1 - exercise 8: write a structure in binary. struct nopad is declared in
 * utils.h
 * so that write_struct and read_struct agree on its layout.
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    struct nopad value = { .c1 = 'a', .l = 5, .c2 = 'b' };
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);
    exit_if(written == -1, "write value");
    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

C tds/td1/read_struct.c

```
/*
 * TD1 - exercise 8: read the structure written by write_struct.
 * ./write_struct | ./read_struct      → { c1 = 'a', l = 5, c2 = 'b' },
 * bytes written = 24
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    struct nopad value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("{ c1 = '%c', l = %ld, c2 = '%c' }, bytes written = %zd\n",
           value.c1, value.l, value.c2, written);
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./write_struct | ./read_struct
{ c1 = 'a', l = 5, c2 = 'b' }, bytes written = 24
```

C tds/td1/struct_sizes.c

```

/*
 * TD1 - exercise 8 (answers): size and padding of struct nopad, and two ways
 * to shrink it.
 * The compiler aligns `long` on 8 bytes: 7 padding bytes after c1 and 7
 * after c2 (arrays of
 * the structure must keep every `l` aligned). -O3 does not change the
 * layout: it is part of
 * the ABI, not an optimisation.
 */
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

struct reordered {          /* biggest member first: only 6 padding bytes at
the end */
    long l;
    char c1;
    char c2;
};

struct __attribute__((packed)) packed { /* no padding at all (slower,
unaligned access) */
    char c1;
    long l;
    char c2;
};

int main(void)
{
    printf("struct nopad      : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct nopad),
        offsetof(struct nopad, c1), offsetof(struct nopad, l),
        offsetof(struct nopad, c2));
    printf("struct reordered : size %2zu  offsets l=%zu c1=%zu c2=%zu\n",
sizeof(struct reordered),
        offsetof(struct reordered, l), offsetof(struct reordered, c1),
        offsetof(struct reordered, c2));
    printf("struct packed     : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct packed),
        offsetof(struct packed, c1), offsetof(struct packed, l),
        offsetof(struct packed, c2));
}

```

```
return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./struct_sizes
struct nopad      : size 24  offsets c1=0 l=8 c2=16
struct reordered  : size 16  offsets l=0 c1=8 c2=9
struct packed     : size 10  offsets c1=0 l=1 c2=9
```

La structure occupe **24 octets** alors que ses champs totalisent 10 : le compilateur aligne `long` sur 8 octets et ajoute 7 octets de remplissage après `c1` et 7 à la fin (pour que, dans un tableau de structures, chaque `l` reste aligné). `-03` ne change rien : la disposition en mémoire fait partie de l'ABI, ce n'est pas une optimisation. Pour la réduire : ordonner les champs du plus grand au plus petit (16 octets) ou la compacter avec `__attribute__((packed))` (10 octets, avec des accès non alignés plus lents et non portables).

⚠ Corrigé par rapport à la version héritée

`read_struct.c` ne compilait pas : la structure n'était pas dans `utils.h` et il utilisait un champ `c` qui n'existe pas.

Liste de vérification

Défi avec l'IA générative · Un `cat` qui mesure ses propres appels système



Objectif. Étendez `read.c` pour qu'il prenne la taille du tampon en argument, mesurez avec `strace -c` et avec le temps d'exécution combien d'appels `read` et `write` il fait pour chaque taille, et expliquez la courbe.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

J'ai ce programme C qui copie l'entrée standard vers la sortie standard avec `read/write` et un tampon de taille fixe [coller `read.c`]. Aide-moi à : (1) prendre la taille du tampon en argument (de 1 octet à 1 Mio) avec `malloc` ; (2) écrire un script bash qui, pour les tailles 1, 16, 256, 4096, 65536 et 1048576, copie un fichier de 50 Mo, mesure le temps avec `/usr/bin/time -f %e` et compte les appels avec `strace -c` ; (3) expliquer pourquoi le temps baisse autant au début puis se stabilise. Ne me donne pas la conclusion avant que j'aie vu les chiffres.

Comment vérifier le résultat

- Le fichier copié est identique (`cmp`) pour toutes les tailles.
- Le nombre d'appels `read` vaut $\approx$ taille du fichier / taille du tampon (plus une dernière lecture qui renvoie 0).
- Vous savez expliquer avec vos mots le coût fixe de chaque appel système (changement de mode).

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.